Retrieval-Augmented Generation:

Baseline Retrieval vs. Cross-Encoder Re-Ranking

Setup summary

Item	Value
Document	CATAN base-game *Game Rules & Almanac* (official rulebook PDF, 16 pages, 15 content pages)
Chunking	sentence-aware, about 700 chars/chunk, 2-sentence overlap, giving **99 chunks** (mean 131 words)
Bi-Encoder	`sentence-transformers/all-MiniLM-L6-v2` (384-dim, cosine)
Cross-Encoder	`cross-encoder/ms-marco-MiniLM-L-6-v2`
LLM	Gemini (`gemini-flash-lite-latest`)
Retrieval	Top-10 retrieved; Top-3 handed to the LLM
Queries	8 sanity queries (easy) and 11 failure queries (stress tests)
Device	Apple Silicon (MPS)
Code	`https://github.com/nikk8076/LMA_RAG_Assignment.git`

A rulebook is a deliberately hard corpus: it is dense with exceptions ("except when…"), negations ("you may not…"), and corner cases whose wording overlaps heavily with unrelated rules. These are precisely the situations where a single-vector dense retriever struggles. On the 8 *sanity* queries (straightforward factual questions such as the cost of a city or the build requirements of a settlement) the baseline already places the correct chunk in its Top-3, so the two pipelines agree. The interesting behaviour is on the 11 *failure* queries below.

1 Architecture Comparison

Bi-Encoder (baseline retriever). A Bi-Encoder encodes each chunk *independently*, ahead of time, into one fixed 384-dimensional vector, and encodes the query the same way at search time. Relevance is the cosine similarity between the query vector and each chunk vector. Because the query and the chunk never interact during encoding, the model must compress each text into a single "topical" point in vector space and then compare points. This is very fast, since all chunk vectors are computed once and stored, and a query becomes a single embedding followed by a dot product against the index. The compression, however, blurs fine distinctions: polarity ("may" vs. "may not"), which specific condition a clause applies to, and whether a shared keyword is actually the subject of a passage or merely mentioned in passing.

Cross-Encoder (re-ranker). A Cross-Encoder takes the query and a candidate chunk *together*: the two texts are concatenated and passed through the transformer as one sequence, so self-attention runs jointly across every token of both. The network reads the pair directly and emits a single relevance score. This recovers exactly what the Bi-Encoder loses, namely term interaction, negation, and which clause answers the question, making it substantially more accurate. The price is that nothing can be pre-computed: the score depends on the specific (query, chunk) pair, so each pair requires a full forward pass at query time. Consequently the

Cross-Encoder is applied only to the small candidate set the cheap Bi-Encoder has already shortlisted. This two-stage design is the standard *retrieve-then-rerank* pattern.

2 Qualitative & Rank-Shift Analysis

I built 11 failure queries, each one chosen so that the chunk that actually answers it is *not* in the Bi-Encoder's Top-3. They span the four failure families named in the assignment: keyword trap, negation and constraint, buried vital chunk, and exception or corner case. In every one of the 11, the Cross-Encoder promotes the answer chunk back into the Top-3.

2.1 Rank shifts

For each failure query the table lists the chunk that answers it, its rank in the baseline Top-10, and its rank after Cross-Encoder re-ranking. In every row the chunk moves from rank 4 or worse into the final Top-3. Across all 11 queries the Cross-Encoder promotes 19 distinct chunks into the Top-3; the table shows the single most relevant promotion per query.

Failure query	**Category**	**Answer chunk**	**Base**	**Re-rank**
What happens when a 7 is rolled?	Keyword trap	64 (discard, move robber, steal)	#4	**#1**
Trade with the bank on another's turn?	Negation	80 ("may not trade with the bank…")	#9	**#1**
Give resource cards away as a gift?	Negation	80 ("you may not give away cards")	#7	#2
Return development cards to the supply?	Negation	29 ("never go back into the supply")	#4	#2
Trade two wool for one wool?	Negation	81 ("may not trade like resources")	#8	#2
Where does the robber start the game?	Buried chunk	63 ("begins the game in the desert")	#7	**#1**
When are victory point cards revealed?	Buried chunk	35 ("only reveal…when the game is over")	#5	#2
Play a knight card before rolling?	Buried chunk	34 ("play the card…even before you roll")	#7	**#1**
Move the robber onto the desert hex?	Exception	43 ("…any other terrain hex or on the desert")	#4	#2
Buy a development card when deck empty?	Exception	19 ("cannot buy…if the supply is empty")	#4	**#1**
Supply runs out while several are owed it?	Exception	14 ("not enough of a given resource…")	#4	#3

2.2 Why the baseline failed (four representative queries)

- **Keyword trap, "What happens when a 7 is rolled?"** The baseline Top-3 were the *Number Tokens* chunk ("…there is no **7**…"), a resource-income chunk, and a set-up chunk, none of which answer the question. The Bi-Encoder was pulled toward the literal token "7", which appears most prominently in the irrelevant number-tokens passage. The real rule (players with more than 7 cards discard half, then the roller moves the robber and steals a card) uses different wording and sat at ranks #4 and #6. The Cross-Encoder read the intent of the question and promoted those chunks to #1 and #2.

- **Negation, "Can I trade with the bank during another player's turn?"** The governing rule is a prohibition ("You may not trade with the bank during another player's turn"), buried at baseline rank #9. The Bi-Encoder instead surfaced generic, positive "trade" passages, whose embeddings sit close to the query but describe when trading *is* allowed. The Cross-Encoder lifted the prohibition to #1.
- **Buried vital chunk, "Where does the robber start the game?"** The baseline filled its Top-3 with robber-*movement* passages (the knight card, "move the robber away…"). The passage stating that the robber *begins* in the desert was at rank #7; re-ranking moved it to #1.
- **Exception / corner case, "Can I buy a development card when the deck is empty?"** The governing clause ("you cannot buy development cards if the supply is empty") sat at rank #4 behind generic "development card" passages; re-ranking promoted it to #1.

The remaining seven failure queries follow the same pattern: the baseline ranks a superficially similar but wrong passage above the answer (for example, positive "trade" or "development card" passages above the specific prohibition), and the Cross-Encoder, which reads the query and chunk jointly, pulls the correct clause into the Top-3.

2.3 Answer 1 vs. Answer 2

Both pipelines use an *identical* prompt template, so any difference in the answer is caused only by which chunks were supplied. The table below gives the actual answers generated by `gemini-flash-lite-latest` for all 11 failure queries: Answer 1 from the baseline Top-3 and Answer 2 from the re-ranked Top-3.

Query	**Answer 1 (baseline Top-3)**	**Answer 2 (re-ranked Top-3)**
What happens when a 7 is rolled?	**Omitted:** "context does not contain the answer"	Full rule: discard half if more than 7 cards, move the robber, steal 1 card
Trade with the bank on another's turn?	**Omitted**	"No… you may not trade with the bank during another player's turn"
Give resource cards away as a gift?	**Omitted**	"No… you may not give away cards"
Return development cards to the supply?	**Omitted**	"Development cards never go back into the supply"
Trade two wool for one wool?	Answered from a nearby chunk	"You may not trade like resources (2 wool for 1 wool)"
Where does the robber start the game?	**Omitted**	"The robber starts the game in the desert"
When are victory point cards revealed?	Answered partially	"Only when the game is over, once a player reaches 10+ VP"
Play a knight card before rolling?	**Omitted**	"Yes… you can play the card even before you roll the dice"
Move the robber onto the desert hex?	Correct (clause already at rank #2)	"Yes… any other terrain hex or the desert hex"
Buy a development card when deck empty?	**Omitted**	"No… you cannot buy development cards if the supply is empty"
Supply runs out while several are owed it?	Answered from a nearby chunk	"No player receives any of that resource that turn"

In 7 of the 11 failure queries the baseline **omitted the answer entirely**, returning "the provided context does not contain the answer" because the critical chunk sat outside its Top-3; a less conservative model could instead have *hallucinated* a plausible but wrong rule. In the other 4 the baseline retrieved a nearby chunk and answered approximately. In every one of the 11, re-ranking promoted the exact answer chunk into the Top-3, so Answer 2 states the precise rule, including the exception or negation wording.

3 Latency Profiling

Measurements are averaged over 5 repeats per query on the MPS device. Baseline cost is the query embedding plus the cosine search over the vector store. Re-ranking adds the Cross-Encoder scoring of all 10 (query, chunk) pairs on top of the baseline. LLM generation time is excluded, as it is network-bound. The one-time index build (embedding all 99 chunks) took **650 ms**.

Failure query	**P1 base (ms)**	**Re-rank (ms)**	**P2 total (ms)**	**Slowdown**
What happens when a 7 is rolled?	5.05	38.33	43.37	8.6×
Trade with the bank on another's turn?	4.89	46.25	51.14	10.5×
Give resource cards away as a gift?	4.86	31.01	35.87	7.4×
Return development cards to the supply?	4.74	30.11	34.85	7.4×
Trade two wool for one wool?	4.48	31.02	35.50	7.9×
Where does the robber start the game?	4.50	30.65	35.15	7.8×
When are victory point cards revealed?	4.66	29.88	34.54	7.4×
Play a knight card before rolling?	4.87	30.39	35.26	7.2×
Move the robber onto the desert hex?	4.76	31.81	36.57	7.7×
Buy a development card when deck empty?	4.82	31.10	35.92	7.5×
Supply runs out while several are owed it?	4.47	31.89	36.35	8.1×
Mean	**4.74**	**32.95**	**37.68**	8.0×

Baseline retrieval is roughly 5 ms because the chunk vectors are pre-computed and the search is a single matrix multiplication. Re-ranking adds about 33 ms, making the end-to-end pipeline roughly **8 times slower**, even though it scores only 10 pairs, because each pair is a full transformer forward pass. That extra latency is the price paid for the accuracy gains shown in Section 2, and on this corpus it is a clearly worthwhile trade.

4 Scaling: why not run the Cross-Encoder over the whole corpus?

Because the Cross-Encoder has nothing to pre-compute. Its score depends jointly on the query and the chunk, so for a new query it would have to run one full transformer forward pass for *every* chunk in the collection, every time. With 99 chunks the Cross-Encoder pass already costs about 33 ms; scaling that to 100k or 1M chunks turns a single query into seconds or minutes of compute. The cost grows linearly with corpus size and is far too slow for interactive retrieval.

The Bi-Encoder avoids this entirely. Chunk embeddings are computed once, offline, and stored in a vector index; a query is then answered with a single embedding plus an approximate-nearest-neighbour lookup in milliseconds, essentially independent of corpus size. The retrieve-then-rerank design exploits both: the cheap Bi-Encoder shortlists a small candidate set (Top-10), and the expensive Cross-Encoder is spent only on those 10 pairs. This keeps the millisecond latency of dense retrieval while recovering most of the Cross-Encoder's accuracy, the benefit that running the Cross-Encoder over the full corpus would be far too costly to obtain directly.